Autonomous Search and Rescue

Project B - IA712 Mobile Robotics, Final Project

Team **RobotZ**

ROS 2 Humble · Ignition Gazebo Fortress · TurtleBot 4 · Nav2 · slam_toolbox · BehaviorTree.CPP

Repository: https://github.com/YiZeems/autonomous-search-and-rescue

Member	Email	Member	Email
Julien GIMENEZ	julien.gimenez@telecom-paris.fr	Paul CINTRA	paul.cintra@telecom-paris.fr
Hugo FANCHINI	hugo.fanchini@telecom-paris.fr	Yimou ZHANG	yimou.zhang@telecom-paris.fr

Abstract. A single TurtleBot 4 autonomously explores an unknown, partitioned disaster arena, builds a map by SLAM, and localizes four "victims" (AprilTags) by projecting camera detections into the global `map` frame with `tf2`. Decision-making is a *Behavior Tree* (not an FSM, as required [1]) that orchestrates a **two-phase mission**: frontier exploration, then map-derived room inspection. The final autonomous run reaches **97.35 % coverage and 4/4 victims** in one click. This report covers the system architecture, the theoretical foundations from the course, the team's decision journey across three representative runs, and the lessons learned.

1 Problem statement and requirements

Search-and-rescue robotics addresses a simple but demanding question: when an environment is too dangerous for people - a collapsed building, a flooded basement, a nuclear accident zone - can a robot go in alone, understand a space it has never seen, and bring back the one piece of information that matters, namely *where the victims are*? This project is a simulated, self-contained answer to that question, and a synthesis of the whole IA712 course: it ties together SLAM, autonomous exploration, navigation and decision-making into a single system that runs from one command and needs no human in the loop.

The scenario [1] places a robot in a simulated disaster zone where humans cannot go. The robot must *explore autonomously* an unknown environment, *localize victims* (AprilTags), and *report their coordinates* in a global frame. What makes the task interesting is precisely that nothing is known in advance: the robot does not have a map, it does not know how many rooms exist or where the victims sit, and it must decide *by itself* where to go next, when it has seen enough, and when the mission is over. The assignment fixes the constraints we satisfy:

Requirement [1]	How `bl` satisfies it
Autonomous exploration (frontier / RRT), *no human input*	Frontier exploration (Yamauchi [2]) with information-gain selection [3], gated by the BT
Map quality, loop closure	`slam_toolbox` async with loop closure [4]
Camera detection, project to `map` via `tf2`	`apriltag_ros` (tag36h11) [7] → `victim_registry` projects camera→`map` (TF2) [8]
Map > 90 % of the area	**97.35 %**
Final map marked with all victim positions	annotated map + 4 victims, Fig. 10
Decision = Behavior Tree, *not* FSM	BehaviorTree.CPP v3 [6]
One-click launch	`ros2 launch rescue_bringup bringup_tb4.launch.py`
Bonus: greedy vs. information-gain coverage	quantitative benchmark (n=3), §3

The system at a glance.

The arena is a partitioned four-room building (`rescue_arena`) with one AprilTag "victim" on an outer wall of each room. The robot is a **TurtleBot 4** (Create 3 base, RPLIDAR A1M8, OAK-D camera) simulated in **Ignition Gazebo Fortress**. From a single launch, the robot maps the building by SLAM, autonomously visits every room, detects the four tags, projects them into the `map` frame, and stops on a clean annotated map - with *no* prior knowledge of the rooms or the victims' locations. The next sections describe the architecture, the course concepts it applies, and how we got there.

2 System architecture

The workspace is split into **four ROS 2 packages** so the four members can work in parallel: `rescue_bringup` (launch + configs), `rescue_robot` (Python "megapackage": exploration, perception, navigation/inspection, results), `rescue_world` (Ignition worlds + AprilTag targets), and `rescue_decision` (the C++ Behavior Tree). Fig. 1 is the detailed system diagram required by [1].

This split is deliberate. A clean separation between *what the robot perceives* (SLAM, perception), *what it does* (exploration, navigation, inspection), *where it acts* (the simulated world) and *how it decides* (the Behavior Tree) let four people work in parallel on their own branches without stepping on each other, and it kept each component testable in isolation - we could exercise the Behavior Tree, the victim registry and the RViz visualisation against lightweight mock publishers, long before the full Gazebo stack existed. Just as importantly, the design draws a firm line between *policy* and *mechanism*: the Behavior Tree decides *which* phase should run and *when* to move on, while the Python nodes are pure executors that know nothing of the overall mission. That line is what makes the system easy to reason about, and it is what the next two sections unfold - first the course concepts the mechanisms rest on, then the mission the policy assembles from them.

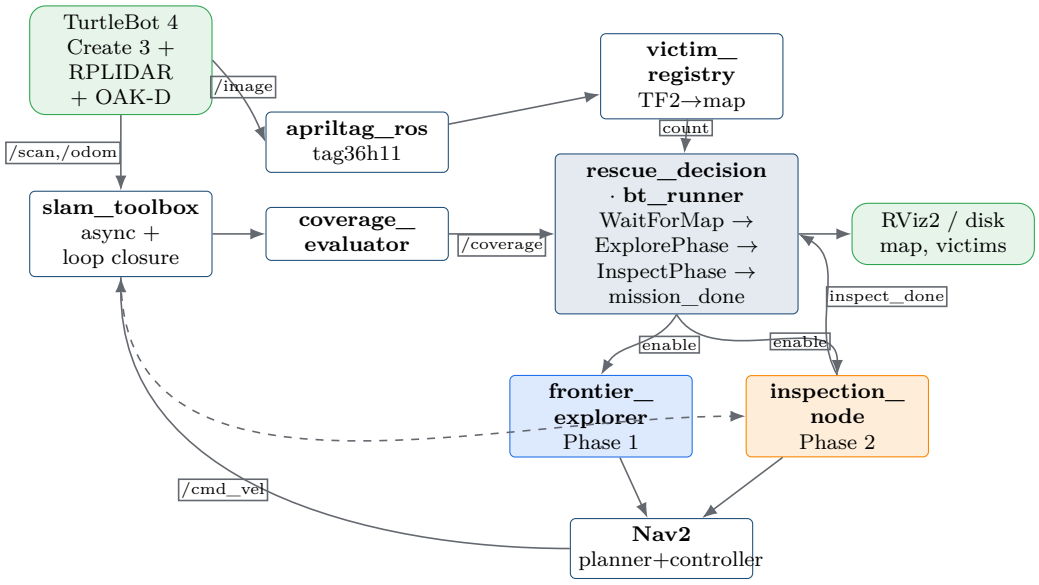


Figure 1: System architecture and topic flow. The Behavior Tree (`rescue_decision`) is the *orchestrator*: it gates the Phase-1 explorer and the Phase-2 inspector via `/mission/*` topics; the Python nodes do the work. SLAM feeds the map to exploration, coverage, Nav2 and inspection.

The packages and their responsibilities:

- **`rescue_bringup`** - the *one-click* entry point (`bringup_tb4.launch.py`) and all configs (Nav2, SLAM, AprilTag). A single launch starts simulation, SLAM, Nav2, perception, the explorer/inspector and the Behavior Tree.
- **`rescue_robot`** - the Python "megapackage": `exploration/` (frontier search + blacklist), `navigation/` (inspection planner + node), `detection/` (victim registry/detector), `results/` (coverage evaluator + result exporter), and dev mocks.
- **`rescue_world`** - the Ignition Fortress arena (`rescue_arena.sdf`) and the voxel AprilTag targets. A *single source* owns victim placement, so the map and the ground truth never diverge.
- **`rescue_decision`** - the real C++ `BehaviorTree.CPP v3` runner (visualisable in Groot), with the mission tree XML. This is the "decision = BT, not FSM" requirement.

Perception pipeline.

Victims are `tag36h11` AprilTags (the assignment explicitly excludes sophisticated human detection / YOLO). A subtle challenge: a PBR-textured tag renders *white* under Ogre2+Mesa, so `apriltag_ros` sees nothing; we build each tag as **voxel boxes** with a white quiet-zone ring around the native 8×8 marker (a naive 10×10 resize destroys the quiet zone and makes the tag *display but stay undetectable*). Each detection yields a `camera`→`victim_`i transform; `victim_registry` projects it to `map` via TF2, deduplicates by ID, and persists `results/victims.json` - the "report their coordinates in the global

frame" requirement.

3 Theoretical foundations (from the course)

SLAM and loop closure.

The robot builds the map online with `slam_toolbox` (pose-graph SLAM). Loop closure - emphasised in the course - corrects accumulated drift when the robot re-observes a known place, keeping the global map consistent during repeated passes [4,9].

Frontier exploration and information gain (CM8).

A *frontier* is the boundary between free and unknown cells [2]. Greedy selection takes the nearest frontier; *information-gain* selection maximises $g(f) = \text{gain}(f) - \lambda \cdot \text{cost}(f)$ [3], where cost is the *Nav2 path length* (not Euclidean). Our n=3 benchmark confirms the theory: info-gain reaches 0.85 ± 0.08 coverage (the only one above 90 %) vs. 0.72 for greedy. A key robustness addition is an *inaccessible-frontier blacklist* keyed by a quantized world coordinate, so a dead frontier never gets reselected even as the grid origin shifts. Concretely, a frontier is blacklisted either when Nav2 fails to reach it, or when it is re-selected **3 times in a row without any coverage gain** (the common case in our arena); it is then never proposed again.

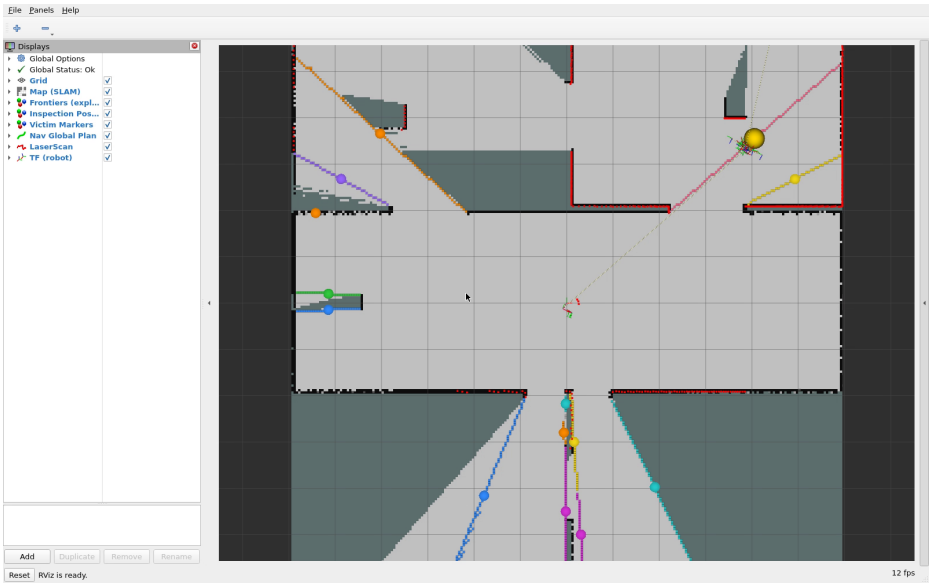


Figure 2: Frontier exploration visualised in RViz (lab/TP style, CM8): the *frontier cells* (the free/unknown boundary) are coloured **per cluster** (connected-component segmentation), the yellow sphere marks the best-utility goal frontier, and the blue line is the matching Nav2 plan. You *see* the coloured rim eat into the unknown up to $> 90\,\%$ coverage (capture: `rviz_capture.mp4`).

Path planning and control (CM9–CM10).

The global planner produces a geometric path; the local controller follows it. We compared *Dijkstra/NavFn* vs. *A* (Smac)* [9]: in a tightly partitioned arena the robot often stands inside the inflation layer, where A* refuses to plan ("start in lethal space"); NavFn tolerates this and is kept. The controller is DWB (sampling-based, CM10).

Coordinate frames and TF2.

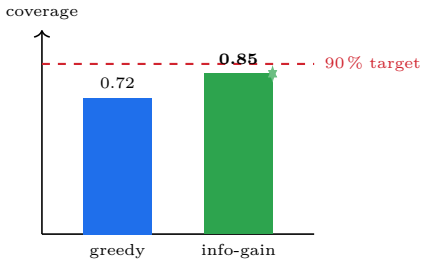
A victim is detected in the *camera* frame; `tf2` chains camera $\rightarrow$ victim_i $\rightarrow$ map to report a global position (Fig. 5b), exactly the "project positions into the global frame" requirement [1,8].

Behavior Trees vs. FSM.

The assignment forbids FSMs. A BT is reactive and composable: a memory `Sequence` ticks children left-to-right, *resumes* at the RUNNING node, and only advances on SUCCESS - a real phase sequence rather than hand-rolled state transitions [6].

The two-phase mission.

Pure exploration completes the *map* but caps victim detection (Fig. 5a). The fix is a second *autonomous* phase: from the map it just built, `inspection_node` derives *one pose per*

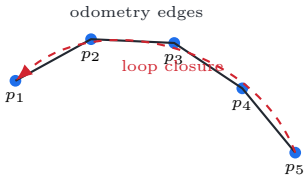


(a) Bonus benchmark (n=3): information-gain is the only strategy above 90 % coverage.

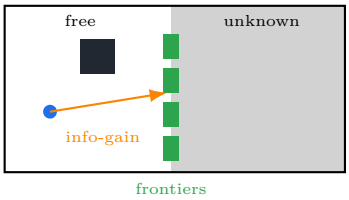
Planner	Property (CM9)	Status in `bl`
Dijkstra/NavFn	optimal, explores all, tolerant	**chosen** (robust)
A* (Smac)	heuristic, faster, strict	tested, rejected here
RRT	fast, non-optimal	outlook

Why NavFn: in a partitioned arena the start is often in the inflation layer; A* aborts there, NavFn does not.

Figure 3: Exploration strategy (bonus) and global-planner comparison, grounded in the course.

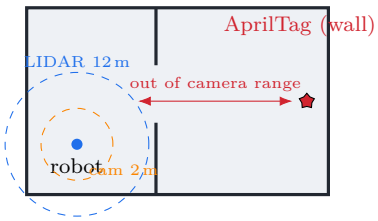


(a) SLAM is a pose graph: re-observing a known place adds a *loop-closure* edge that corrects accumulated drift and keeps the global map consistent across repeated passes.

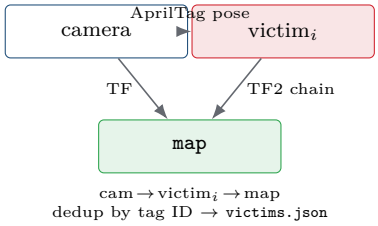


(b) A *frontier* is the free/unknown boundary (green); information-gain selects the most informative one, scored by Nav2 *path* cost (not Euclidean).

Figure 4: Two course concepts: pose-graph SLAM with loop closure (CM6–7) and frontier exploration with information gain (CM8).



(a) Camera-vs-LIDAR gap: LIDAR maps a room from the doorway, but the camera never gets within 2 m of the wall tag ⇒ pure exploration misses victims.



(b) TF2 projection of a detection into the global `map` frame.

Figure 5: Two core concepts behind the mission.

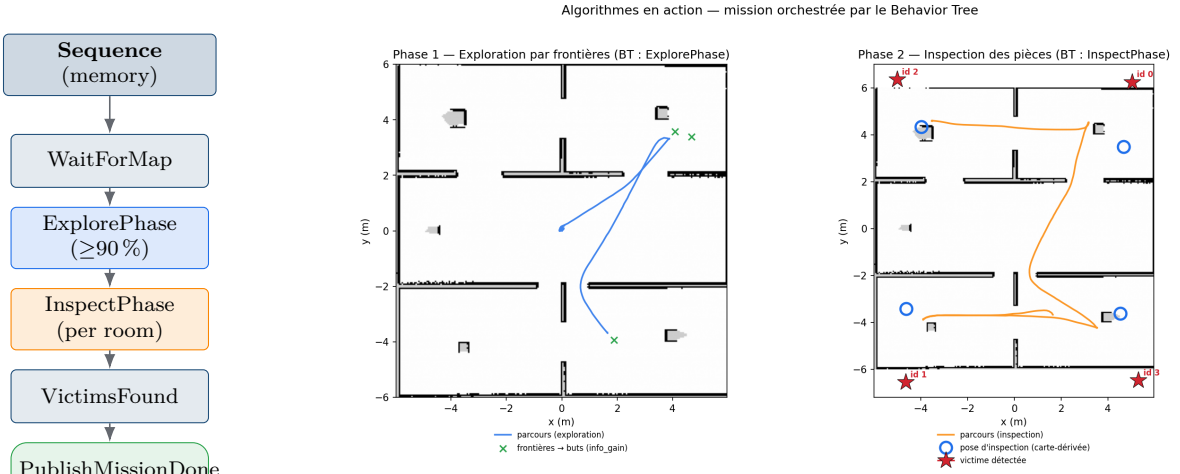
discovered room (facing the nearest wall, snapped to a navigable cell, *no victim coordinates*); the robot drives there and sweeps the camera. The BT mission tree is the Sequence WaitForMap → ExplorePhase → InspectPhase → VictimsFound → mission_done (Fig. 6).

Course concepts *applied* in the project.

Table 1 maps each lecture topic to where it lives in our system - the project is essentially the course made concrete.

Course concept	**Where it is applied in `bl`**
SLAM + loop closure (CM6–7)	`slam_toolbox` (async): online `/map` + `map`→`odom` TF, loop closure on revisits
Frontier exploration (CM8)	`frontier_explorer_node`: free/unknown boundary, world-keyed blacklist
Information gain (CM8, bonus)	info-gain frontier selection; quantitative benchmark vs. greedy (n=3)
Path planning (CM9)	Nav2 global planner - NavFn (Dijkstra) kept; A* (Smac) compared
Local control (CM10)	Nav2 DWB controller (sampling-based trajectory scoring)
Coordinate frames / TF2	`victim_registry`: camera→`victim`$_i$ →`map` projection
Behavior Trees (decision)	`rescue_decision` (BT.CPP v3): the two-phase mission orchestration

Table 1: The course concepts, and exactly where each one is applied in the project.



(a) Mission tree: a memory `Sequence` that resumes the RUNNING phase.

(b) Algorithms in action. *Left:* Phase 1 frontier exploration (blue path, green info-gain goals). *Right:* Phase 2 inspection (orange path, map-derived poses) catching the four victims. Walls are drawn over the path, so it visibly threads the doorways.

Figure 6: The two-phase mission orchestrated by the Behavior Tree.

4 Decision journey: three representative runs

The headline numbers - 97 % coverage, four victims, one click - hide a long and instructive road. None of it came in one shot: almost every capability we now take for granted was, at some point, a run that did the wrong thing for a reason we did not yet understand. We kept a written decision log in the *Problem* → *Investigation* → *Decision* → *Why* format, and the single most useful habit it taught us was to *measure before prescribing* - to let an event log or a metric, rather than an intuition, tell us what was actually wrong. Three runs distill that story; each is reported below as what *blocked* and what we *improved*, and Fig. 7 is the one-page overview.

Run 1 - pure autonomous exploration (v20–v22).

Our first complete system did the obvious thing: frontier exploration driving Nav2, with `slam_toolbox` building the map underneath. It worked, in the sense that it autonomously mapped some ninety percent of the building - but it kept finding only about two of the four victims, and for a long time we could not see why. The answer turned out to be geometric rather than algorithmic: a twelve-metre LIDAR maps a room comfortably from its doorway, so a robot optimising for *coverage* has no reason to walk up to a wall, yet the camera only recognises a tag from about two metres. Exploration was therefore *complete* and detection *structurally incomplete* at the same time. Along the way this run also taught us two sharp lessons - a frontier the planner can never reach will trap the robot in an infinite loop unless it is blacklisted, and a real-time factor low enough to starve the GPU camera renderer will silently stop detection while the camera's metadata keeps flowing and hides the fault. We left this run with a clean map and dependable autonomy, and a clear diagnosis of what was still missing.

Run 2 - two-phase, BT-orchestrated (v32).

The fix followed directly from that diagnosis: if the robot will not approach the walls while exploring, give it a second reason to. We added an *inspection* phase that reads the freshly built map, derives one pose per discovered room about 1.3 m from the nearest wall - inside camera range, but using only the map and never the victims' positions - and visits each in turn while sweeping the camera. Crucially, we also promoted the Behavior Tree from a passive supervisor that merely watched the coverage metric into the genuine *orchestrator* of the mission, gating exploration and inspection in sequence; this is what makes the system conformant to the "decision by Behavior Tree, not FSM" requirement in substance and not just in name. The result was the breakthrough we had been chasing: **four victims out of four and 97 % coverage** in one continuous, fully autonomous run.

Run 3 - the final deliverable.

With the mission working, the last run was about turning a result into a deliverable that is

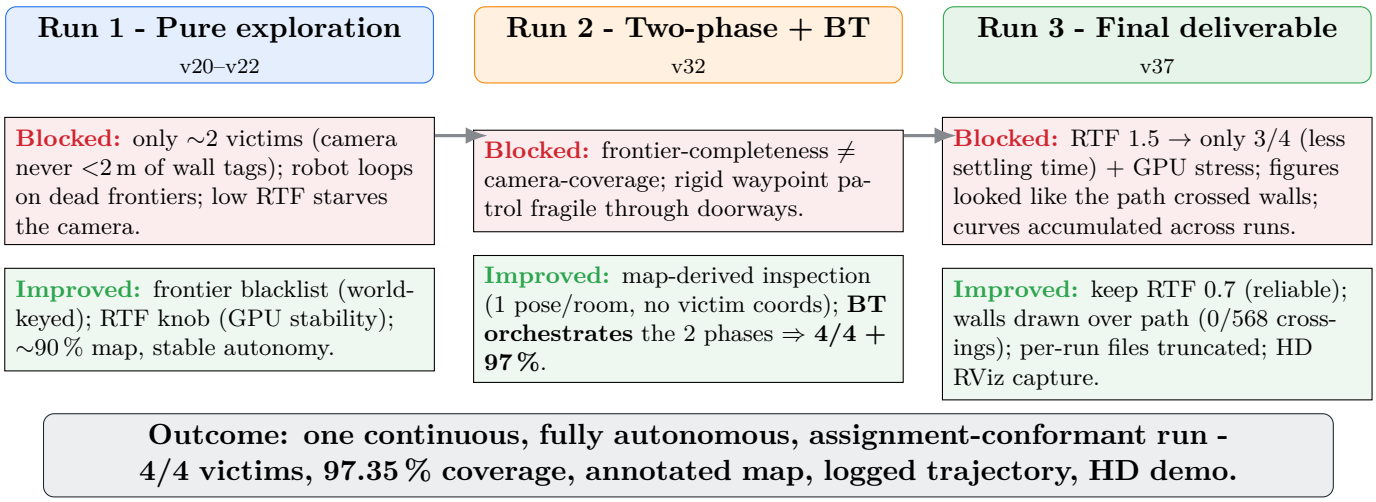


Figure 7: The decision journey on one page: three runs, each as *what blocked* (red) → *what we improved* (green). Two distinct root causes were unmasked late - the Nav2 cost function discouraged entering rooms (fixed coverage), and a collapsed RTF starved the camera renderer (fixed detection); the two-phase Behavior Tree then made 4/4 reliable.

trustworthy and reproducible. We first asked whether we could simply run the simulation faster: pushing the real-time factor to 1.5 did finish sooner, but it gave the camera less time to settle at each pose and quietly dropped us back to three victims out of four, besides stressing the GPU - so we kept the slower, reliable factor, a small reminder that for a deliverable, repeatability outweighs speed. We then closed a set of presentation-grade issues that, while not affecting the robot's behaviour, affected whether a reader could *trust* the figures: the logged map-frame trajectory is now drawn with the walls painted *over* it, so the path is only ever visible in free space and visibly threads each doorway (we verified that not one of its 568 segments crosses a wall); the occupancy map is rendered with the correct vertical orientation, which had been the real reason an earlier figure *looked* as though the robot drove through walls; and each run's result files are truncated on start, so the curves of one run no longer pile up on top of another's. The robustness work that lets the inspection recover an unreachable pose on a teammate's machine (§3) also landed here. The outcome is the run this entire report draws on: **four victims out of four, about 97 % coverage**, a clean annotated map, a logged trajectory, and an HD capture of the live RViz algorithms - a single, reproducible source for every figure shown.

Challenges and solutions in detail

Reaching 4/4 inside Run 1–2 meant peeling six *stacked* causes - each fix revealed the next (Table 2). The decisive insight: *two* independent root causes hid under the symptoms - the Nav2 cost function discouraged the robot from *entering* rooms (a **coverage** problem, fixed by lowering the inflation radius and cost scaling), and a collapsed real-time factor *starved the camera renderer* while `camera_info` kept flowing (a **detection** problem, very misleading). The two-phase Behavior Tree then made the result reliable.

#	Symptom	Root cause	Decision (fix)
1	Robot immobile, 0 victim	Patrol waypoints in *unknown* space	Hybrid: explore first, *then* act
2	Waypoints rejected/blocking	Goals land on a wall	Validate goals in free space
3	"Failed to make progress"	Create 3 safety reflex	`safety_override=full`
4	Deep corners rejected at 93 %	Corner cells still unknown	Goal-snapping to nearest free cell
5	Inter-room routing fragile	No global path at goal time	Pivot: 360° spin-and-scan
6	Only 1 victim detected	Low RTF starves camera renderer	Raise RTF; camera off while exploring

Table 2: Six stacked causes behind "0 victims", and the decision taken at each step.

Host stability (WSL/GPU) - measure before prescribing.

Long runs rebooted the Windows host. "Obvious" culprits were wrong *twice*: memory was fine

(`free`: 5/19 GiB) and no Windows Update was pending. The event log revealed the real causes: GPU `VIDEO_TDR_FAILURE` / `CLOCK_WATCHDOG` bugchecks under sustained load. Our controllable fix is the **real-time-factor knob** (`IA712_GZ_RT_RATE`): staying on the GPU (software GL is ∼23× slower) but lowering the physics/render cadence reduces sustained load without changing *simulated* time - so coverage and `time_to_90` are unaffected.

5 Team collaboration and task distribution

We worked on per-member `dev/*` branches integrated into `dev/bl` (Fig. 8); `bl` is the merged version this report describes.

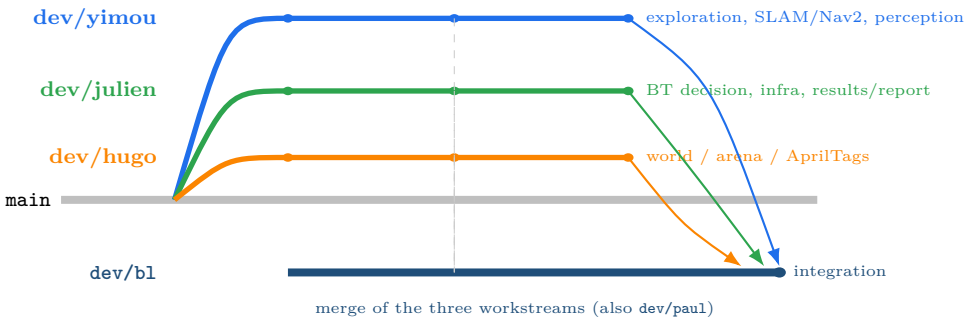


Figure 8: Branch construction: three per-developer feature branches (workstreams from the git history) are merged into `dev/bl`, the integration branch.

Task distribution (from the git history): **Yimou ZHANG** - autonomous exploration, SLAM/Nav2 wiring, perception core, benchmark; **Julien GIMENEZ** - Behavior-Tree decision & two-phase orchestration, simulation infrastructure (WSL/GPU/RTF/DDS), results, integration (`bl`) and report; **Hugo FANCHINI** - simulation world, rescue arena and AprilTag target placement (`rescue_world`); **Paul CINTRA** - perception/detection support and testing (`dev/paul`).

6 Results

Fig. 9 shows the live RViz algorithms across the mission; Fig. 10 the deliverable artefacts. All come from the single final run (`results/examples/l18_nominal`).

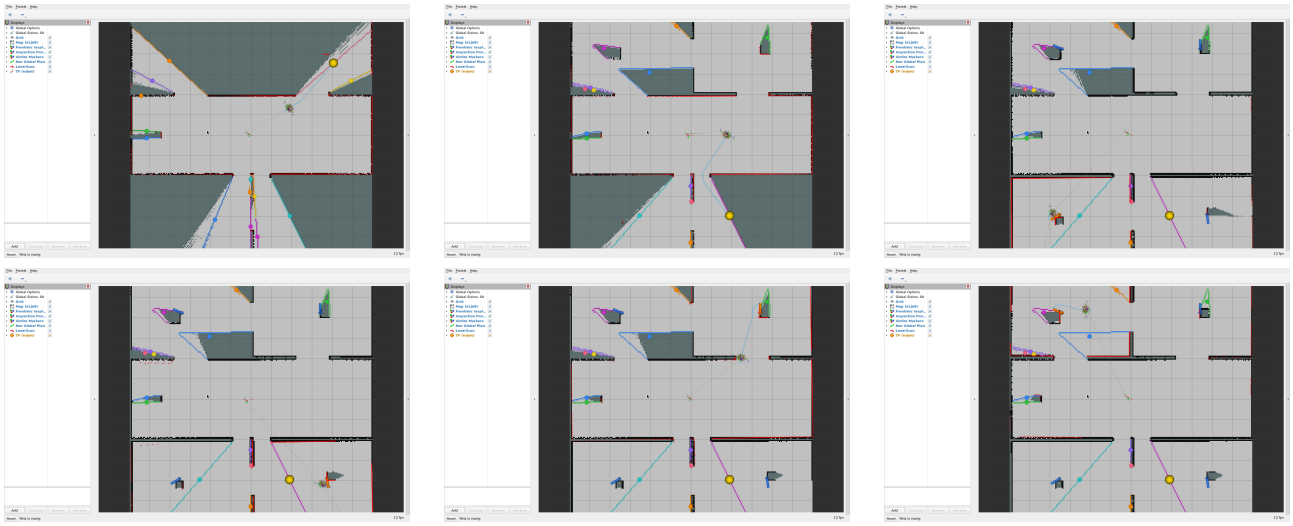
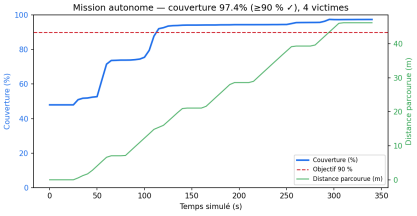


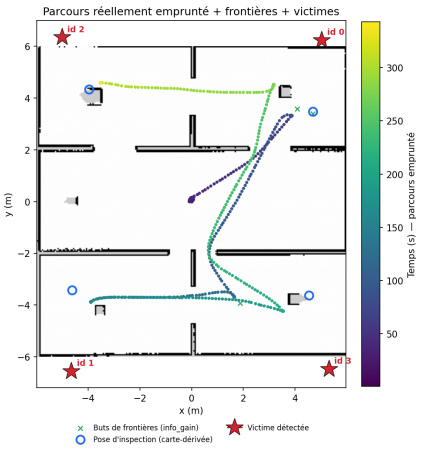
Figure 9: Live RViz at six key moments (1–3: early exploration, map nearly built, phase transition; 4–6: room inspection, victims appearing, final map) - showing the map (SLAM), frontiers, inspection poses, Nav2 plan, LaserScan and the four victim markers. Captured at HD on a virtual X server.

Reading the results.

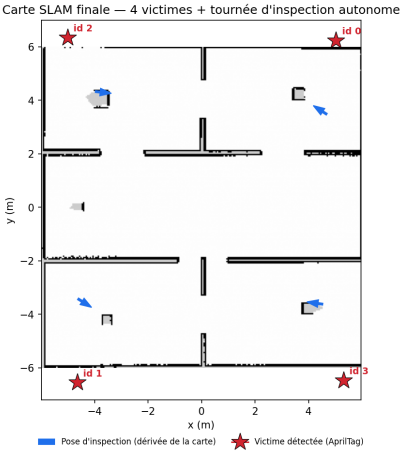
The coverage curve (Fig. 10a) shows the two phases clearly: a fast rise during exploration up to the 90 % target, then a plateau during inspection while the path length keeps growing (the robot still moves, room to room, but the map is already complete). The mission timeline (Fig. 11a) overlays the four victim-detection instants on the phase bands: the first victims are caught *during* exploration



(a) Coverage & path vs. time (90 % target).

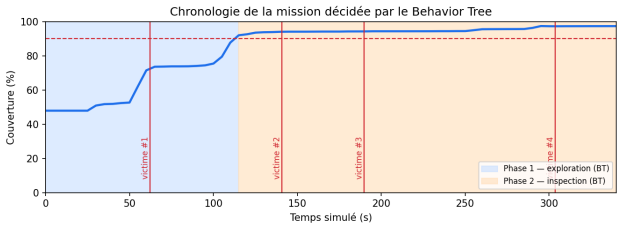


(b) Real path (time-coloured), threading doorways.

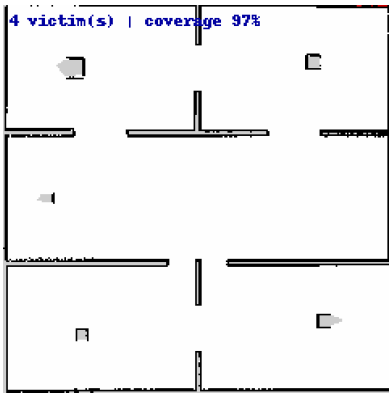


(c) Final map: 4 victims + inspection tour.

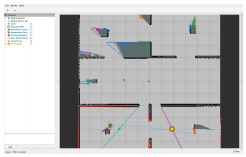
Figure 10: Quantitative results: **97.35 %** coverage and **4/4** victims, fully autonomous. The trajectory is the actual map-frame path; walls are drawn over it, so it visibly passes through doorways (verified: no segment crosses a wall).



(a) BT-decided timeline: Phase 1 (blue) / Phase 2 (orange) bands and the instant each victim is detected.



(b) Deliverable: final map marked with the victims.



(c) Live RViz.

Figure 11: Mission chronology and the assignment deliverable ("final map marked with all victim positions"). The phase split and victim-detection instants confirm the BT orchestration.

when the robot happens to pass close to a wall tag, and the remaining ones *during* inspection - exactly the behaviour the two-phase design predicts. The trajectory (Fig. 10b) confirms the robot visits all four rooms and returns, threading every doorway.

Running and reproducibility.

The whole stack starts with one command, `ros2 launch rescue_bringup bringup_tb4.launch.py`. The nominal run is archived (`results/examples/l18_nominal`) so every figure in this report is regenerable from data; the bonus benchmark is *resumable* (each run persists its summary, valid runs are skipped on restart), which is how the campaign survived the host reboots. RViz is captured to HD video on a headless machine via a virtual X server, giving the live-algorithm sequence of Fig. 9.

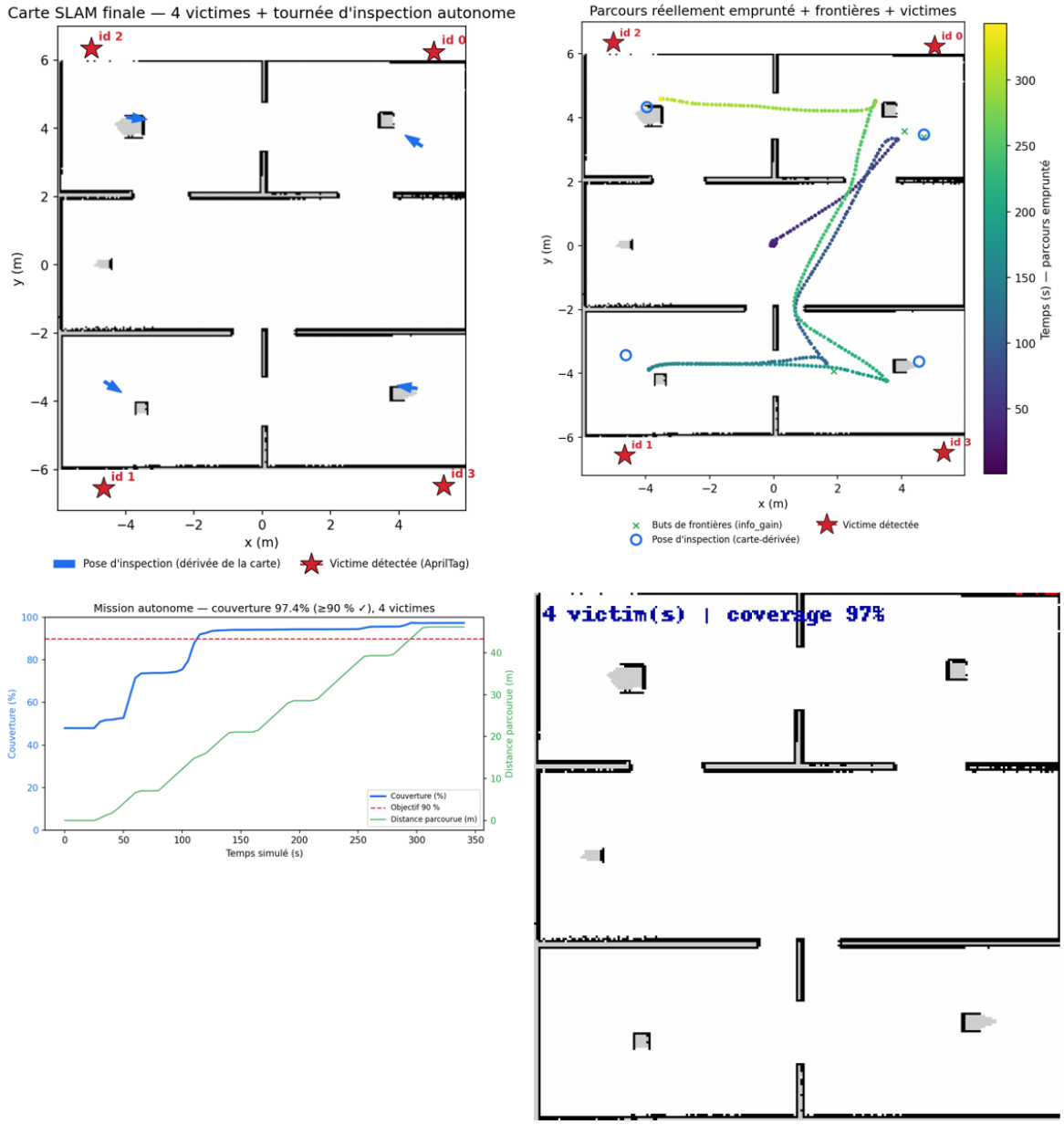


Figure 12: One-glance summary montage: final map with the four victims and the inspection tour (top-left), the real time-coloured trajectory (top-right), coverage and path length over time (bottom-left), and the deliverable annotated map (bottom-right). Everything is regenerated from the single archived run, so the figures are reproducible from data.

7 Lessons learned and timeline

Lessons learned.

Four lessons stand out, and they are as much about engineering method as about robotics. The first is to *measure before prescribing*. Twice we were certain we knew why the host machine kept rebooting - it had to be memory, then surely a pending Windows Update - and twice we were wrong; only the event log, `free` and `nvidia-smi` pointed at the real culprits, a GPU thermal-driver crash and, separately, a camera renderer starved by a collapsed real-time factor while its metadata kept flowing and hid the problem. The second lesson is that *one fix reveals the next*: getting from zero to four victims meant peeling six stacked causes - an unmapped goal, a goal on a wall, a safety reflex, an unreachable corner, fragile inter-room routing, and finally the starved camera - where each correction simply exposed the one beneath it. The third is that *robust beats rigid*: in a partitioned arena, an adaptive explorer that already visits every room, followed by a full 360° camera sweep, consistently beats a hand-authored waypoint patrol that breaks the moment the map differs slightly. And the fourth is *conformity by design* - by making the Behavior Tree truly *orchestrate* the mission rather than merely watch it, we satisfy the assignment's "decision by BT, not FSM" requirement in spirit and not only on paper.

Timeline (L13–L18).

The project followed the suggested schedule (Fig. 13).

Figure 13: The six sessions L13–L18 and their milestones.

8 Discussion and conclusion

Looking back, the project succeeds less because of any single clever algorithm than because of how the pieces are made to cooperate. The exploration is a textbook frontier search, the planner and controller are stock Nav2 plugins, the SLAM is `slam_toolbox` out of the box, and the detector is `apriltag_ros`; what we contributed is the *integration* - a Behavior Tree that turns these independent components into a coherent mission, and a second autonomous phase that closes the gap between "the map is complete" and "every victim is found". This is exactly the shift the assignment asks for: from writing code that merely runs to engineering a system whose parts hold together under real conditions.

The two-phase design is the conceptual heart of the result. A robot that only explores will, in a partitioned building, map each room comfortably from its doorway with a twelve-metre LIDAR and never need to step inside - which is efficient for mapping but fatal for a camera that only sees a tag from two metres. Separating *mapping* from *inspection*, and deriving the inspection poses from the map the robot has just built rather than from any prior knowledge of the victims, keeps the system fully autonomous while making detection reliable. The Behavior Tree is what makes this separation clean: each phase is a node that runs until its own success condition, and the mission simply reads as a sequence.

Robustness, finally, was a lesson in itself. A run that works on one machine is not the same as a system that works: a slightly different costmap on a teammate's computer was enough to make two inspection poses unreachable, until the inspector learned to wait for the costmap to settle and to fall back to a pose pulled into the room's interior. The honest conclusion of this project is therefore as much about method - measure, isolate one cause at a time, make the system tolerant rather than rigid - as it is about the final numbers. Natural next steps would be to compare A* against NavFn on a less aggressive costmap, to try a Regulated Pure Pursuit controller for smoother doorway traversals, and to add a second robot and let the Behavior Tree coordinate a shared map.

Acknowledgements

We warmly thank **Zhi Yan**, Teacher-Researcher at ENSTA, for the IA712 course and guidance that made this project possible.

References

[1] IA712 Mobile Robotics - Final Project, Project B (assignment). Course site: https://yzrobot.github.io/IA712/.

[2] B. Yamauchi, *A frontier-based approach for autonomous exploration*, IEEE CIRA, 1997.

[3] C. Stachniss et al., *Information gain-based exploration using Rao-Blackwellized particle filters*, ICRA, 2005.

[4] S. Macenski, I. Jambrecic, *slam_toolbox: SLAM for the dynamic world*, JOSS, 2021.

[5] S. Macenski et al., *The Marathon 2: A navigation system (Nav2)*, IROS, 2020.

[6] D. Faconti et al., *BehaviorTree.CPP* library, https://www.behaviortree.dev/.

[7] E. Olson, *AprilTag: A robust and flexible visual fiducial system*, ICRA, 2011; `apriltag_ros`.

[8] T. Foote, *tf2: The transform library*, IEEE TePRA, 2013.

[9] IA712 lecture notes (CM6–CM10: SLAM, Exploration, Path Planning, Control), https://yzrobot.github.io/IA712/.

[10] Project repository, https://github.com/YiZeems/autonomous-search-and-rescue.